



Seguridad informática aplicada a Entornos industriales

**Diseño e implementación de una arquitectura
DevSecOps basada en GitOps:
Aseguramiento de la cadena de suministro en
Kubernetes mediante scanners, ArgoCD y
Kyverno**

Memòria

**Ziad El Karrabi El Hanafi
Tutor: Josep Andreu**

2026

Índice general

Índice de figuras	III
Índice de cuadros	V
Glosario	VII
1. Introducción	1
2. Marco teórico y análisis de referentes	3
2.1. Evolución del paradigma DevOps a DevSecOps	3
2.1.1. Limitaciones de los modelos tradicionales	3
2.1.2. Integración de la seguridad en el ciclo de vida del software	3
2.1.3. Nacimiento del concepto DevSecOps	3
2.2. Kubernetes como plataforma de ejecución	4
2.2.1. Arquitectura de Kubernetes	4
2.2.2. Componentes principales del clúster	4
2.2.3. Gestión de red y Network Policies	4
2.3. Policy-as-Code y Admission Controllers	5
2.3.1. Funcionamiento de los Admission Controllers	5
2.3.2. Validación vs mutación de recursos	5
2.3.3. Importancia de la seguridad declarativa	5
2.4. Herramientas y tecnologías utilizadas	6
2.4.1. Kyverno	6
2.4.2. OPA Gatekeeper	6
2.4.3. ArgoCD y GitOps	6
2.4.4. GitHub Actions	7
2.4.5. Trivy y Sysdig Secure	7
2.5. Comparativa teórica Kyverno vs OPA	7
3. Objetivos y alcance	9
3.1. Objetivo general	9
3.2. Objetivos específicos	9
3.3. Alcance del proyecto	10
4. Metodología del proyecto	11
4.1. Enfoque general del proyecto	11
4.2. Entorno de laboratorio	11
4.3. Fases del proyecto	12
4.3.1. Fase 1: Aprovisionamiento del clúster Kubernetes	12
4.3.2. Fase 2: Despliegue e integración de Kyverno	12
4.3.3. Fase 3: Implementación de políticas de seguridad	13
4.3.4. Fase 4: Integración GitOps con ArgoCD	13
4.3.5. Fase 5: Integración CI/CD con GitHub Actions y Trivy	13
4.4. Selección y alcance de las políticas de seguridad	13

4.5.	Reporting y observabilidad del sistema	14
4.6.	Estrategia de validación	15
4.7.	Comparativa con OPA Gatekeeper	15
4.8.	Decisiones de alcance y limitaciones previstas	16
5.	Diseño de la arquitectura	17
5.1.	Arquitectura general del sistema	17
5.2.	Descripción de componentes	18
5.2.1.	Clúster Kubernetes	18
5.2.2.	Kyverno	19
5.2.3.	ArgoCD	19
5.2.4.	GitHub Actions y Trivy	20
5.2.5.	Sysdig Secure	20
5.3.	Flujo de despliegue DevSecOps	21
5.3.1.	Fase de integración continua	21
5.3.2.	Fase de despliegue continuo y admisión	21
5.4.	Arquitectura de red del clúster	23
5.5.	Decisiones de diseño	24
5.5.1.	Separación entre CI y CD	24
5.5.2.	Alcance de ClusterPolicy frente a Policy	25
5.5.3.	Modo Enforce frente a Audit	25
5.5.4.	Mutación antes de validación como estrategia de hardening	25
5.5.5.	synchronize: true en políticas de generación	26
5.5.6.	Seguridad de la cadena de suministro del pipeline	26
5.5.7.	Elección de Calico como plugin CNI	26
	Bibliografía	27

Índice de figuras

1.1. Arquitectura general del entorno DevSecOps	2
5.1. Arquitectura general del entorno DevSecOps	18
5.2. Flujo de integración y despliegue continuo con análisis de seguridad	21
5.3. Flujo de admisión de Kyverno: mutación antes de validación	22
5.4. Arquitectura de red del clúster de laboratorio	23

Índice de cuadros

2.1. Comparativa entre Kyverno y OPA Gatekeeper	8
---	---

Glosario

API REST

Interfaz de Programación de Aplicaciones basada en el paradigma *Representational State Transfer* (REST), que utiliza el protocolo HTTP para facilitar la comunicación entre sistemas y permite realizar operaciones como consultar, crear, modificar y eliminar recursos de manera escalable y eficiente.

Ataque a la cadena de suministro

Tipo de ataque que compromete el software o las herramientas utilizadas durante el proceso de desarrollo o despliegue, en lugar de atacar directamente el sistema objetivo. En entornos CI/CD, puede manifestarse como la manipulación de dependencias, imágenes de contenedores, actions de GitHub o runners de terceros para inyectar código malicioso en el pipeline.

Auto-hardening

Proceso automático de aplicación de configuraciones de seguridad sobre recursos del sistema sin intervención manual del operador o desarrollador. En el contexto de este trabajo, se implementa mediante políticas de mutación de Kyverno que inyectan campos de seguridad en los manifiestos de Kubernetes en el momento del despliegue.

CNI

(*Container Network Interface*) Especificación y conjunto de bibliotecas que definen cómo los plugins de red deben configurar las interfaces de red de los contenedores en Kubernetes. Ejemplos de plugins CNI son Calico, Flannel y Cilium.

Drift protection

Mecanismo que detecta y corrige automáticamente cualquier desviación entre el estado deseado de un recurso (definido declarativamente) y su estado real en el sistema. En el contexto de Kyverno, se implementa mediante el campo `synchronize: true` en las políticas de tipo *generate*, que regenera automáticamente los recursos eliminados o modificados de forma no autorizada.

GitOps

Paradigma operativo que utiliza repositorios Git como fuente única de verdad para definir y gestionar el estado deseado de la infraestructura y las aplicaciones. Cualquier

cambio en el sistema se realiza mediante commits en el repositorio, y un operador automatizado (como ArgoCD) se encarga de sincronizar el estado real del entorno con el estado declarado en Git.

IMDS

(*Instance Metadata Service*) Servicio disponible en entornos cloud (AWS, GCP, Azure) accesible desde cualquier instancia a través de la IP de enlace local `169.254.169.254`. Expone información sensible de la instancia como credenciales IAM temporales, tokens de servicio y configuración de red. Es un vector de ataque habitual en entornos de contenedores comprometidos.

PolicyReport

Recurso nativo de Kubernetes generado automáticamente por Kyverno que consolida los resultados de la evaluación de políticas sobre los recursos del clúster. Indica qué recursos han pasado, fallado o sido mutados por cada política, con marca temporal de cada evaluación. Existen dos variantes: `PolicyReport` (con alcance de namespace) y `ClusterPolicyReport` (con alcance de clúster).

1 Introducción

La adopción de arquitecturas basadas en contenedores y orquestadores como Kubernetes ha transformado la forma en que se desarrollan y despliegan aplicaciones en entornos cloud y corporativos. Este paradigma permite automatizar la gestión del ciclo de vida de las aplicaciones, mejorar la escalabilidad y facilitar la integración de prácticas de desarrollo modernas. Sin embargo, esta evolución tecnológica también introduce nuevos desafíos en materia de seguridad, especialmente en entornos donde los despliegues se realizan de forma automatizada y continua.

En este contexto surge el enfoque DevSecOps, cuyo objetivo es integrar la seguridad dentro del propio ciclo de vida del desarrollo de software, en lugar de tratarla como una fase posterior al despliegue. Este modelo promueve la automatización de controles de seguridad dentro de los pipelines de integración y despliegue continuo, permitiendo detectar vulnerabilidades, aplicar políticas de seguridad y garantizar configuraciones seguras desde las primeras fases del desarrollo.

El presente Trabajo de Fin de Máster se centra en la implementación de un entorno DevSecOps completo sobre Kubernetes, integrando herramientas de Integración Continua (GitHub Actions) con análisis automatizado de vulnerabilidades mediante herramientas como Trivy y Sysdig Secure. Asimismo, se implementará un modelo de Despliegue Continuo basado en GitOps utilizando ArgoCD, permitiendo gestionar la infraestructura y los despliegues de forma declarativa a través de repositorios Git.

Adicionalmente, se incorporará el uso de Kyverno como motor de políticas de seguridad basado en Policy-as-Code, permitiendo aplicar controles automáticos sobre los manifiestos de Kubernetes durante el proceso de despliegue.

El núcleo del proyecto se basa en el uso de Admission Controllers, concretamente Kyverno, para aplicar políticas de seguridad de forma declarativa dentro del clúster de Kubernetes. A diferencia de los enfoques tradicionales que únicamente bloquean configuraciones inseguras, este trabajo explorará el uso de políticas de tipo mutating, capaces de corregir automáticamente configuraciones inseguras en los manifiestos YAML en tiempo de despliegue.

Finalmente, el proyecto incluirá un análisis comparativo entre Kyverno y OPA Gatekeeper, evaluando sus capacidades en la implementación de políticas de seguridad en Kubernetes, su facilidad de uso, curva de aprendizaje y eficacia en entornos de producción.

La Figura 1.1 muestra una visión general de la arquitectura DevSecOps implementada en este trabajo, donde se integran los procesos de desarrollo, análisis de seguridad, despliegue continuo y aplicación de políticas dentro de un entorno Kubernetes.

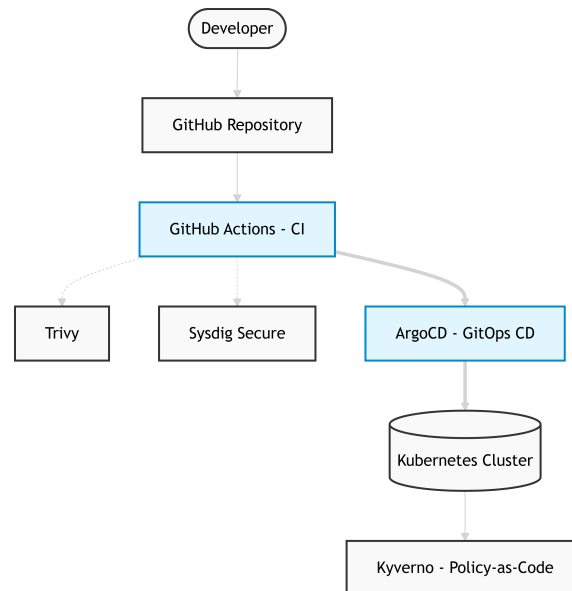


Figura 1.1: Arquitectura general del entorno DevSecOps

2 Marco teórico y análisis de referentes

2.1 Evolución del paradigma DevOps a DevSecOps

2.1.1 Limitaciones de los modelos tradicionales

Los modelos tradicionales de desarrollo de software, como el modelo en cascada, se caracterizan por una separación estricta de fases (análisis, desarrollo, pruebas y despliegue), lo que dificulta la detección temprana de errores y vulnerabilidades [1]. En estos enfoques, la seguridad se introduce habitualmente en etapas finales, incrementando significativamente el coste de corrección y el riesgo en producción.

La adopción de metodologías ágiles y, posteriormente, del paradigma DevOps permitió mejorar la colaboración entre equipos y acelerar los ciclos de entrega [2]. Sin embargo, diversos estudios destacan que la seguridad continuó tratándose como un proceso independiente, generando vulnerabilidades especialmente en entornos cloud-native [3].

En este contexto, se hace evidente la necesidad de integrar mecanismos de seguridad automatizados que puedan aplicarse de forma continua dentro del ciclo de vida del software.

2.1.2 Integración de la seguridad en el ciclo de vida del software

La evolución hacia DevSecOps implica la integración de la seguridad desde las primeras fases del desarrollo, incorporando controles automatizados en pipelines CI/CD [4]. Esto incluye análisis de vulnerabilidades, validación de configuraciones y control de cumplimiento.

La automatización permite garantizar consistencia y reducir errores humanos, especialmente en entornos dinámicos. Según [5], la integración temprana de la seguridad mejora significativamente la resiliencia del software.

Este enfoque resulta clave en el presente trabajo, donde la seguridad se implementa mediante mecanismos automatizados integrados en Kubernetes.

2.1.3 Nacimiento del concepto DevSecOps

DevSecOps surge como una evolución natural de DevOps, integrando la seguridad como un componente intrínseco del ciclo de vida del software [6]. Este paradigma promueve la automatización de controles de seguridad y la responsabilidad compartida entre equipos.

En entornos basados en contenedores y Kubernetes, este enfoque requiere mecanismos capaces de aplicar políticas de seguridad de forma dinámica y escalable.

En este trabajo, este objetivo se materializa mediante el uso de Policy-as-Code y herramientas como Kyverno, que permiten aplicar seguridad de forma declarativa y automatizada.

2.2 Kubernetes como plataforma de ejecución

2.2.1 Arquitectura de Kubernetes

Kubernetes se ha consolidado como el estándar de facto para la orquestación de contenedores, proporcionando una plataforma robusta para la gestión de aplicaciones distribuidas [7].

Su arquitectura se basa en un plano de control (control plane), que gestiona el estado del sistema, y nodos worker encargados de ejecutar las cargas de trabajo. El API Server actúa como punto central de interacción mediante un modelo declarativo [8].

Esta arquitectura permite automatizar la gestión del sistema, pero también introduce nuevos retos de seguridad que deben ser abordados mediante mecanismos específicos.

2.2.2 Componentes principales del clúster

Los componentes principales del control plane incluyen el API Server, el Scheduler y el Controller Manager, responsables de procesar solicitudes y mantener el estado deseado del sistema. En los nodos worker, el kubelet y el runtime de contenedores permiten la ejecución de los pods [8].

La existencia de múltiples componentes distribuidos hace necesario implementar controles de seguridad centralizados, como los Admission Controllers.

2.2.3 Gestión de red y Network Policies

Por defecto, Kubernetes permite la comunicación libre entre pods, lo que puede representar un riesgo significativo. Las Network Policies permiten definir reglas de comunicación basadas en etiquetas y namespaces, aplicando el principio de mínimo privilegio [8].

Además, en entornos reales es habitual encontrar configuraciones inseguras como:

- Contenedores ejecutándose como usuario root

- Escalada de privilegios habilitada
- Falta de límites de recursos
- Acceso no restringido a red

Estas prácticas suponen un riesgo elevado, especialmente en entornos multi-tenant.

Esto justifica la necesidad de aplicar controles automáticos en el momento del despliegue mediante políticas de seguridad en Kubernetes.

2.3 Policy-as-Code y Admission Controllers

2.3.1 Funcionamiento de los Admission Controllers

Los Admission Controllers son mecanismos del API Server que interceptan las solicitudes antes de que los recursos sean desplegados en el clúster [8]. Permiten aplicar validaciones o modificaciones sobre los objetos.

Este mecanismo constituye la base técnica sobre la que se implementará la seguridad en este trabajo.

2.3.2 Validación vs mutación de recursos

Los *validating admission controllers* permiten verificar que los recursos cumplen ciertas políticas, rechazando aquellos que no las satisfacen.

Por otro lado, los *mutating admission controllers* permiten modificar automáticamente los recursos antes de su creación, aplicando configuraciones seguras por defecto.

En este trabajo se dará especial importancia a las políticas de tipo *mutating*, ya que permiten aplicar mecanismos de hardening automático sobre los manifiestos, corrigiendo configuraciones inseguras sin intervención del usuario.

Este enfoque resulta clave para garantizar la seguridad en entornos DevSecOps altamente automatizados.

2.3.3 Importancia de la seguridad declarativa

El enfoque Policy-as-Code permite definir políticas de seguridad como código versionado, facilitando su integración en flujos GitOps [9].

Este modelo mejora la trazabilidad, reproducibilidad y automatización de la seguridad.

En este trabajo, estos mecanismos serán implementados mediante Kyverno para aplicar políticas automáticamente en tiempo de despliegue.

2.4 Herramientas y tecnologías utilizadas

2.4.1 Kyverno

Kyverno es una herramienta de Policy-as-Code diseñada específicamente para Kubernetes que permite definir políticas utilizando YAML, sin necesidad de aprender lenguajes adicionales [10].

A diferencia de otras soluciones, Kyverno permite definir políticas directamente en el mismo formato utilizado por Kubernetes, lo que facilita su adopción. Además, soporta tanto políticas de validación como de mutación, permitiendo no solo bloquear configuraciones inseguras, sino también corregirlas automáticamente.

Esta capacidad resulta especialmente relevante en entornos DevSecOps, donde la automatización y la consistencia son factores clave.

Kyverno constituye el eje central de este trabajo.

2.4.2 OPA Gatekeeper

Open Policy Agent (OPA) es un motor de políticas generalista que permite definir reglas mediante el lenguaje Rego [11]. Gatekeeper facilita su integración con Kubernetes.

Aunque ofrece gran flexibilidad, su complejidad y enfoque basado en validación lo diferencian de Kyverno.

2.4.3 ArgoCD y GitOps

ArgoCD implementa el paradigma GitOps, donde el estado del sistema se define en repositorios Git [12].

En este trabajo, se utilizará para automatizar el despliegue y garantizar la coherencia entre configuración y estado real.

2.4.4 GitHub Actions

GitHub Actions permite automatizar pipelines CI/CD, incluyendo procesos de build, test y seguridad [13].

Se utilizará para integrar controles de seguridad dentro del flujo DevSecOps.

2.4.5 Trivy y Sysdig Secure

Trivy permite el escaneo de vulnerabilidades en imágenes de contenedores [14], mientras que Sysdig Secure proporciona seguridad en tiempo de ejecución [15].

Estas herramientas complementan la seguridad aplicada mediante políticas en Kubernetes.

2.5 Comparativa teórica Kyverno vs OPA

Las herramientas Kyverno y Open Policy Agent (OPA) representan dos de las soluciones más relevantes dentro del enfoque *Policy-as-Code* aplicado a Kubernetes. Ambas permiten definir y aplicar políticas de seguridad sobre los recursos del clúster, aunque presentan diferencias significativas en cuanto a su diseño, enfoque y capacidades [10], [11].

Kyverno es una herramienta diseñada específicamente para Kubernetes (*Kubernetes-native*), que permite definir políticas utilizando el mismo formato YAML empleado en los manifiestos del propio sistema. Este enfoque reduce la complejidad y facilita su adopción por parte de equipos DevOps familiarizados con Kubernetes. Por el contrario, OPA utiliza el lenguaje Rego, un lenguaje declarativo de propósito general que ofrece una gran flexibilidad, pero introduce una mayor curva de aprendizaje.

Desde el punto de vista funcional, ambas herramientas permiten implementar políticas de validación (*validating*), es decir, comprobar que los recursos cumplen determinadas condiciones antes de ser desplegados. Sin embargo, una diferencia clave radica en el soporte de políticas de mutación (*mutating*). Kyverno permite modificar automáticamente los recursos en tiempo de admisión, aplicando configuraciones seguras por defecto, mientras que OPA Gatekeeper se centra principalmente en la validación, careciendo de soporte nativo para la mutación de recursos.

A continuación, se presenta una comparativa resumida de ambas herramientas:

Característica	Kyverno	OPA Gatekeeper
Enfoque	Kubernetes-native	Generalista
Lenguaje de políticas	YAML	Rego
Curva de aprendizaje	Baja	Alta
Validación de recursos	Sí	Sí
Mutación de recursos	Sí	No (limitada)
Integración con Kubernetes	Nativa	Mediante CRDs
Facilidad de adopción	Alta	Media

Cuadro 2.1: Comparativa entre Kyverno y OPA Gatekeeper

Estas diferencias tienen un impacto directo en su aplicabilidad en entornos DevSecOps. Mientras que OPA ofrece una mayor expresividad y flexibilidad para definir políticas complejas, Kyverno destaca por su simplicidad operativa y su capacidad de integrarse de forma natural en flujos de trabajo basados en Kubernetes.

Desde la perspectiva de este trabajo, la capacidad de mutación de Kyverno resulta especialmente relevante, ya que permite implementar mecanismos de *hardening* automático sobre los manifiestos, corrigiendo configuraciones inseguras sin necesidad de intervención manual. Esto incluye, por ejemplo, la adición automática de límites de recursos, la desactivación de privilegios elevados o la aplicación de políticas de red por defecto.

En consecuencia, Kyverno se posiciona como una herramienta especialmente adecuada para entornos donde se prioriza la automatización, la consistencia y la integración con prácticas GitOps.

Esta comparativa teórica servirá como base para el análisis práctico que se desarrollará en capítulos posteriores, donde se evaluará el comportamiento de ambas soluciones en un entorno real de Kubernetes.

3 Objetivos y alcance

3.1 Objetivo general

El objetivo principal de este Trabajo de Fin de Máster es el diseño e implementación de un entorno funcional de DevSecOps sobre Kubernetes, integrando mecanismos de seguridad de forma nativa dentro del ciclo de vida del desarrollo y despliegue de aplicaciones.

El núcleo del proyecto se centra en la aplicación del paradigma Policy-as-Code mediante el uso de Admission Controllers, concretamente Kyverno, con el objetivo de garantizar configuraciones seguras en tiempo de despliegue. A diferencia de los enfoques tradicionales basados únicamente en la detección o el bloqueo, este trabajo pone especial énfasis en el uso de políticas de tipo *mutating*, capaces de corregir automáticamente configuraciones inseguras en los manifiestos de Kubernetes.

Esta aproximación permite no solo prevenir errores de configuración, sino también establecer mecanismos de hardening automático, alineados con las prácticas modernas de seguridad en entornos cloud-native.

3.2 Objetivos específicos

Para alcanzar el objetivo general, se definen los siguientes objetivos específicos:

- Desplegar un clúster Kubernetes funcional en un entorno de laboratorio, con una arquitectura basada en un nodo de control y un nodo worker, configurando adecuadamente su comunicación y la capa de red mediante una solución CNI como Calico.
- Instalar y configurar Kyverno como motor de políticas de seguridad basado en Policy-as-Code, integrándolo como Admission Controller dentro del clúster.
- Definir e implementar un conjunto representativo de políticas de seguridad (entre 5 y 10), centradas en problemáticas reales como:
 - ejecución de contenedores con privilegios elevados,
 - escalada de privilegios,
 - ausencia de límites de recursos,
 - control de comunicaciones de red.

- Validar el comportamiento de las políticas mediante pruebas prácticas, demostrando tanto el bloqueo de configuraciones inseguras (*validating policies*) como su corrección automática (*mutating policies*).
- Implementar una política avanzada basada en la generación automática de Network Policies, con el objetivo de restringir accesos a recursos sensibles, como servicios de metadatos (por ejemplo, la IP 169.254.169.254), habitual en entornos cloud.
- Integrar el clúster dentro de un modelo de despliegue continuo basado en GitOps, utilizando ArgoCD para gestionar el estado declarativo de las aplicaciones.
- Incorporar un pipeline de integración continua mediante GitHub Actions, incluyendo procesos de construcción y análisis de vulnerabilidades con herramientas como Trivy o Sysdig Secure.
- Realizar una comparativa técnica entre Kyverno y OPA Gatekeeper, analizando diferencias en facilidad de uso, capacidades de validación y mutación, y adecuación en entornos productivos.

3.3 Alcance del proyecto

El proyecto se desarrolla en un entorno de laboratorio basado en máquinas virtuales, con el objetivo de reproducir un escenario realista de despliegue en Kubernetes. El alcance incluye la configuración del clúster, la integración de las herramientas principales del pipeline DevSecOps y la implementación de políticas de seguridad representativas.

No obstante, el trabajo no pretende cubrir de manera exhaustiva todos los estándares de seguridad existentes, sino demostrar el funcionamiento y la viabilidad del modelo Policy-as-Code aplicado a Kubernetes mediante casos prácticos.

Asimismo, la utilización de herramientas de automatización como Ansible se considera una posible ampliación del proyecto. Siguiendo un enfoque iterativo, se prioriza inicialmente la implementación de los mecanismos de seguridad y su validación, dejando la automatización completa del despliegue como una línea de trabajo futura o paralela en función del tiempo disponible.

4 Metodología del proyecto

El presente capítulo describe el enfoque metodológico adoptado para llevar a cabo el Trabajo de Fin de Máster. No se trata de un capítulo técnico, sino de una descripción razonada de cómo se ha planificado el trabajo, qué decisiones se han tomado respecto al alcance y la organización del proyecto, y qué criterios han guiado la selección de herramientas y la definición de las fases de desarrollo. Muchas de estas decisiones han sido consensuadas con el tutor, Josep Andreu, a lo largo de distintas reuniones y comunicaciones que han servido para ajustar progresivamente el enfoque del trabajo.

4.1 Enfoque general del proyecto

El proyecto adopta un enfoque de **investigación aplicada con desarrollo práctico**. El objetivo no es realizar una revisión teórica exhaustiva de los estándares de seguridad en Kubernetes, sino construir un entorno funcional real que demuestre cómo integrar la seguridad de forma automatizada dentro del ciclo de vida del desarrollo y el despliegue. Los resultados se validan empíricamente a través de pruebas controladas sobre el entorno implementado, no mediante análisis bibliográfico.

El tutor orientó el trabajo hacia un número manejable de entre cinco y diez políticas de seguridad bien elegidas, con el argumento de que demostrar el funcionamiento y la comprensión profunda de Kyverno es más valioso académicamente que aplicar un estándar completo de forma superficial. Esta directriz ha marcado toda la planificación del proyecto: se prioriza la calidad y la justificación de cada decisión sobre la cantidad de elementos implementados.

El desarrollo sigue un modelo **iterativo e incremental**: se comienza con los componentes más fundamentales (el clúster y Kyverno), se valida cada elemento antes de añadir el siguiente, y se avanza progresivamente hacia los componentes de mayor complejidad (ArgoCD, GitHub Actions, comparativa con OPA). Este modelo permite detectar problemas de integración de forma temprana y garantiza que el entorno es funcional en todo momento, incluso antes de estar completo.

4.2 Entorno de laboratorio

El entorno sobre el que se desarrolla el proyecto consiste en un clúster Kubernetes desplegado sobre dos máquinas virtuales proporcionadas por el tutor, gestionadas con VirtualBox. Esta decisión fue consensuada durante el intercambio inicial con Josep Andreu, quien re-

comendó utilizar las imágenes de las máquinas empleadas en la asignatura para garantizar coherencia y estabilidad.

Las dos máquinas virtuales tienen los siguientes roles y direcciones IP fijas dentro de la red del laboratorio:

- **Nodo de control** (*control plane*): `192.168.1.101`, hostname `control`. Aloja el plano de control de Kubernetes y actúa como punto de entrada para todas las operaciones de administración del clúster.
- **Nodo worker**: `192.168.1.102`, hostname `worker`. Ejecuta las cargas de trabajo desplegadas en el clúster.

Cada máquina virtual está configurada con dos interfaces de red: una de tipo NAT para el acceso a internet, y una de tipo *Host-only Adapter* para la comunicación interna entre nodos. La capa de red del clúster se implementa mediante **Calico v3.28** [16], un plugin CNI que permite definir y aplicar efectivamente Network Policies, lo cual es un requisito para una de las políticas de seguridad implementadas en el proyecto.

La elección de un entorno virtualizado local, en lugar de un proveedor cloud, responde a criterios de control y reproducibilidad. No obstante, esta decisión tiene implicaciones sobre el alcance del trabajo, que se detallan al final de este capítulo.

4.3 Fases del proyecto

El proyecto se organiza en cinco fases secuenciales. Cada fase tiene un alcance delimitado y no se avanza a la siguiente hasta verificar que la anterior es funcional.

4.3.1 Fase 1: Aprovisionamiento del clúster Kubernetes

La primera fase comprende la puesta en marcha del clúster Kubernetes sobre las dos máquinas virtuales del laboratorio. Incluye la inicialización del nodo de control, la incorporación del nodo worker y la instalación del plugin de red Calico. El resultado esperado es un clúster operativo con ambos nodos activos y todos los componentes del sistema en funcionamiento.

4.3.2 Fase 2: Despliegue e integración de Kyverno

La segunda fase introduce Kyverno como motor de políticas de seguridad dentro del clúster. Kyverno se instala como un conjunto de controladores que se integran en el mecanismo de

Admission Controllers de Kubernetes, interceptando las solicitudes al API Server para aplicar las políticas definidas. Esta fase también incluye la verificación de que los webhooks de Kyverno están correctamente registrados y operativos antes de proceder con la implementación de políticas.

4.3.3 Fase 3: Implementación de políticas de seguridad

La tercera fase constituye el núcleo del trabajo. Se diseña e implementa un conjunto de políticas de seguridad que cubren los tres tipos de acción disponibles en Kyverno: validación, mutación y generación. Las políticas se aplican de forma incremental, comenzando por las más simples (validación de configuraciones inseguras) y avanzando hacia las de mayor complejidad (mutación automática de manifiestos y generación de recursos derivados).

4.3.4 Fase 4: Integración GitOps con ArgoCD

Una vez que las políticas están validadas y operativas, la cuarta fase introduce ArgoCD para gestionar el estado del clúster de forma declarativa a través de un repositorio Git. Todas las configuraciones — políticas de Kyverno, manifiestos de prueba y configuraciones de infraestructura — se almacenan en Git, y ArgoCD garantiza que el estado del clúster se mantiene sincronizado con el repositorio. Esta fase demuestra el ciclo GitOps completo: cualquier cambio en el repositorio se propaga automáticamente al clúster, y cualquier desviación manual es detectada y revertida.

4.3.5 Fase 5: Integración CI/CD con GitHub Actions y Trivy

La quinta fase incorpora la dimensión de integración continua al entorno DevSecOps. Se configura un pipeline en GitHub Actions que ejecuta automáticamente un análisis de vulnerabilidades sobre las imágenes de contenedores utilizadas, empleando Trivy como motor de escaneo. Esta fase cierra el ciclo DevSecOps completo e introduce además la consideración de los riesgos de la propia cadena de suministro del pipeline, motivada por la alerta del tutor sobre el ataque del grupo TeamPCP, que comprometió pipelines CI/CD mediante la manipulación de actions y runners de terceros [17].

4.4 Selección y alcance de las políticas de seguridad

El conjunto de políticas implementadas no pretende cubrir de forma exhaustiva todos los controles de seguridad existentes para Kubernetes. Siguiendo la orientación del tutor, se han seleccionado entre cinco y diez políticas que ilustran los diferentes mecanismos de acción de Kyverno y que corresponden a problemáticas de seguridad reales y bien documentadas.

Las políticas seleccionadas abordan las siguientes categorías de riesgo:

- Ejecución de contenedores con el usuario root
- Escalada de privilegios en tiempo de ejecución
- Uso de imágenes procedentes de registros no autorizados
- Ausencia de límites de recursos (CPU y memoria)
- Configuración insegura del contexto de seguridad del contenedor
- Acceso de escritura al sistema de ficheros del contenedor
- Acceso al servicio de metadatos de instancia cloud (IP 169.254.169.254)

La última política — el bloqueo del acceso al servicio de metadatos cloud — fue propuesta explícitamente por el tutor como ejemplo ilustrativo del mecanismo de generación de Kyverno. Mediante una política de tipo *generate*, Kyverno crea automáticamente una Network-Policy en cada namespace nuevo del clúster que impide el acceso a dicha IP. Si la Network-Policy es eliminada manualmente, Kyverno la regenera de forma automática, demostrando el concepto de protección frente a desviaciones de configuración (*drift protection*).

Adicionalmente, el tutor señaló el interés de incorporar validación de imágenes mediante firma digital con herramientas como Cosign o Notary, así como el uso del contexto `imageRegistry` de Kyverno para inspeccionar metadatos de imágenes. Estas funcionalidades se contemplan como una posible ampliación del trabajo si el tiempo disponible lo permite, pero no forman parte del alcance comprometido.

4.5 Reporting y observabilidad del sistema

El tutor indicó expresamente que el reporting es un valor añadido relevante para el proyecto. Kyverno genera automáticamente recursos `PolicyReport` y `ClusterPolicyReport` que consolidan los resultados de la evaluación de políticas sobre todos los recursos del clúster. Estos informes permiten conocer, de forma centralizada, qué recursos cumplen cada política, cuáles han sido mutados automáticamente y cuáles han sido rechazados.

La revisión y documentación de estos informes forma parte del plan de validación del sistema y se incluirá como evidencia en el capítulo de resultados. Este mecanismo aporta una dimensión de auditoría y compliance que complementa las pruebas funcionales individuales y demuestra la aplicabilidad del sistema en entornos reales donde la trazabilidad de las evaluaciones de seguridad es un requisito.

4.6 Estrategia de validación

La validación del sistema se realiza mediante pruebas controladas sobre el entorno de laboratorio. Para cada política implementada se ejecuta al menos un escenario de prueba que demuestra su comportamiento con evidencia documental directa: salidas de comandos, estados del sistema y capturas del entorno.

Las pruebas se organizan en tres categorías:

- **Pruebas de bloqueo:** se intenta desplegar un recurso que infringe una política de validación y se verifica que el API Server lo rechaza con el mensaje de error correspondiente.
- **Pruebas de mutación:** se despliega un recurso con una configuración incompleta o mínima y se comprueba que Kyverno ha aplicado automáticamente los campos de seguridad esperados, sin que el desarrollador haya tenido que especificarlos.
- **Pruebas de generación:** se crea un namespace y se verifica tanto la generación automática de la NetworkPolicy asociada como su regeneración tras un borrado manual, demostrando el mecanismo de *drift protection*.

Adicionalmente, se ejecuta una prueba integral que despliega un pod con configuración mínima y verifica que el conjunto de políticas de mutación activas lo transforma en un pod con perfil de seguridad completo. Esta prueba constituye la demostración más representativa del concepto central del trabajo: el *auto-hardening* automático y transparente para el desarrollador.

4.7 Comparativa con OPA Gatekeeper

El proyecto incluye una comparativa práctica entre Kyverno y OPA Gatekeeper. Para ello, se reimplementarán en OPA un subconjunto representativo de las políticas ya definidas en Kyverno, con el objetivo de ilustrar las diferencias en sintaxis, capacidades y facilidad de uso. La comparativa no busca determinar cuál herramienta es superior en términos absolutos, sino evaluar cuál se adapta mejor a las necesidades concretas de un entorno DevSecOps automatizado donde la mutación y la generación de recursos son requisitos clave.

Este ejercicio práctico complementa la comparativa teórica del capítulo 2 y permite extraer conclusiones fundamentadas en la experiencia directa con ambas herramientas.

4.8 Decisiones de alcance y limitaciones previstas

A lo largo del proceso de planificación, se han tomado varias decisiones explícitas sobre el alcance del proyecto que conviene documentar:

Sysdig Secure. El tutor advirtió que Sysdig Secure es un servicio SaaS que ha presentado históricamente problemas de integración con máquinas virtuales, requiriendo en ocasiones el uso de infraestructura cloud para funcionar correctamente. Se contempla su integración si resulta viable con el entorno de laboratorio, pero se acepta de antemano que puede ser necesario documentar la limitación y sustituirla por un análisis de las capacidades de seguridad en tiempo de ejecución que la herramienta ofrecería en un entorno cloud real.

Ansible. Aunque existe interés en incorporar Ansible para automatizar el provisionamiento del clúster y la instalación de los componentes, el tutor recomendó abordarlo únicamente si el resto del trabajo está completo. Ansible se contempla como una ampliación opcional que no condiciona la entrega principal del proyecto.

Validación avanzada de imágenes. Las funcionalidades de verificación de firmas digitales (Cosign, Notary) y de inspección de metadatos mediante `imageRegistry` quedan fuera del alcance comprometido, aunque se incluirán si el tiempo lo permite. Su ausencia no afecta a la demostración del modelo Policy-as-Code, que queda suficientemente cubierta por el conjunto de políticas principal.

Estas decisiones reflejan una planificación realista orientada a garantizar un entregable sólido dentro del tiempo disponible, en lugar de abarcar un alcance excesivo con riesgo de quedarse incompleto.

5 Diseño de la arquitectura

El presente capítulo describe el diseño arquitectónico del entorno DevSecOps implementado en este trabajo. Se expone la visión general del sistema, el papel de cada componente, el flujo completo de integración y despliegue continuo con seguridad integrada, la topología de red del clúster de laboratorio y, finalmente, las decisiones de diseño que han guiado la construcción del entorno y las razones técnicas que las justifican.

5.1 Arquitectura general del sistema

La arquitectura del sistema se organiza en tres planos funcionales claramente diferenciados: el plano de desarrollo, el plano de automatización y el plano de ejecución. Esta separación no es meramente conceptual, sino que refleja una división real de responsabilidades entre los componentes del sistema y los momentos del ciclo de vida en los que actúa cada uno.

El **plano de desarrollo** abarca el conjunto de actividades realizadas por el equipo de ingeniería: la escritura de código de aplicación, la definición de manifiestos Kubernetes y la formulación de políticas de seguridad. Toda esta producción se materializa en forma de commits sobre repositorios Git, que constituyen la fuente de verdad del sistema.

El **plano de automatización** comprende los mecanismos que procesan los cambios introducidos en los repositorios y transforman el código en artefactos desplegados, verificando su seguridad en el proceso. Este plano engloba el pipeline de integración continua gestionado por GitHub Actions, que construye las imágenes de contenedor y las somete a análisis de vulnerabilidades mediante Trivy, y el controlador GitOps implementado por ArgoCD, que detecta las diferencias entre el estado deseado definido en Git y el estado real del clúster y aplica las reconciliaciones necesarias.

El **plano de ejecución** corresponde al clúster Kubernetes, donde las cargas de trabajo finalmente se despliegan y donde Kyverno actúa como última línea de defensa, interceptando cada solicitud de creación o modificación de recursos para aplicar las políticas de seguridad definidas antes de que cualquier objeto sea persistido en el sistema.

La Figura 5.1 ilustra la arquitectura global del sistema, mostrando la relación entre los tres planos y los flujos de información entre ellos.

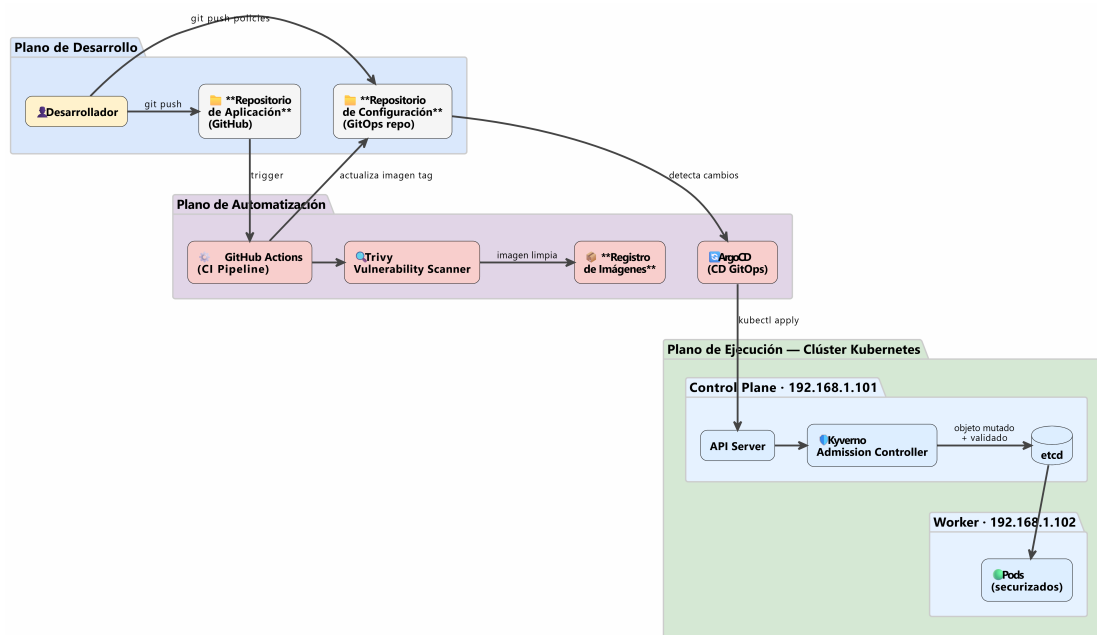


Figura 5.1: Arquitectura general del entorno DevSecOps

Un aspecto arquitectónico central es la adopción del **patrón de dos repositorios** para la implementación del modelo GitOps. El repositorio de aplicación contiene el código fuente de las cargas de trabajo y la lógica de negocio, mientras que el repositorio de configuración —denominado habitualmente *GitOps repo*— almacena los manifiestos de Kubernetes, las políticas de Kyverno y las configuraciones de infraestructura. Esta separación permite que el pipeline de CI modifique únicamente el repositorio de configuración (actualizando la etiqueta de imagen tras una build exitosa) sin que ArgoCD deba tener acceso al código fuente de la aplicación, reduciendo la superficie de ataque y clarificando las responsabilidades de cada componente.

5.2 Descripción de componentes

5.2.1 Clúster Kubernetes

El clúster Kubernetes constituye la plataforma de ejecución sobre la que se sustenta el conjunto del sistema. Siguiendo la arquitectura descrita en el Capítulo 4, se compone de dos nodos: un nodo de control (192.168.1.101) y un nodo worker (192.168.1.102), ambos ejecutando Ubuntu Server 24.04 LTS sobre máquinas virtuales VirtualBox.

El **API Server** es el componente central del plano de control y el único punto de entrada para todas las operaciones sobre el clúster. Todas las herramientas del sistema —ArgoCD, Kyverno, `kubectl`— interactúan exclusivamente a través del API Server, que expone una

API RESTful y aplica los mecanismos de autenticación, autorización y control de admisión sobre cada solicitud recibida [8]. Es precisamente en este componente donde se registran los webhooks de Kyverno para interceptar el tráfico de admisión.

El almacén **etcd** persiste el estado completo del clúster en forma de pares clave-valor distribuidos. Ningún recurso llega a etcd sin haber pasado por el proceso completo de admisión, lo que garantiza que el estado persistido siempre cumple las políticas de seguridad definidas.

5.2.2 Kyverno

Kyverno se despliega en el namespace dedicado `kyverno` y opera mediante cuatro controladores independientes, cada uno con responsabilidades bien delimitadas [10]:

El **Admission Controller** es el componente crítico para este trabajo. Se registra en el API Server mediante dos objetos de tipo `MutatingWebhookConfiguration` y `ValidatingWebhookConfiguration`, que instruyen al API Server para que derive las solicitudes de admisión hacia Kyverno antes de persistirlas. Este controlador evalúa las políticas de tipo *mutate* y *validate* en tiempo real, con una latencia que debe mantenerse por debajo del umbral de timeout configurado en el webhook (por defecto, 10 segundos).

El **Background Controller** evalúa políticas sobre recursos ya existentes en el clúster. Esto es especialmente relevante para las políticas de tipo *generate*: cuando se aplica la política `generate-networkpolicy-metadata`, el Background Controller se encarga de generar retroactivamente las NetworkPolicies en los namespaces que ya existían antes de que la política fuera desplegada.

El **Reports Controller** genera y mantiene actualizados los recursos `PolicyReport` y `ClusterPolicyReport`, que consolidan los resultados de la evaluación de todas las políticas sobre todos los recursos del clúster. Estos informes son fundamentales para la auditoría de cumplimiento.

El **Cleanup Controller** gestiona la eliminación programada de recursos generados por políticas de tipo *generate* cuando el recurso que los originó es eliminado, manteniendo la coherencia del clúster.

5.2.3 ArgoCD

ArgoCD implementa el paradigma GitOps actuando como operador de reconciliación continua entre el estado declarado en el repositorio de configuración y el estado real del clúster [18]. Su arquitectura interna se compone de varios subcomponentes: el *Application Contro-*

ller es el núcleo que detecta las diferencias de estado (*diff*) y aplica las reconciliaciones; el *Repo Server* gestiona las conexiones con los repositorios Git y la renderización de los manifiestos; y el *API Server* expone la interfaz de usuario y la API de gestión.

En el contexto de este trabajo, ArgoCD gestiona las políticas de Kyverno como cualquier otro recurso Kubernetes: los objetos `ClusterPolicy` almacenados en el repositorio de configuración se sincronizan automáticamente al clúster. Esto significa que cualquier modificación de una política —corrección, endurecimiento o nueva política— sigue el mismo flujo de aprobación versionada que cualquier cambio de infraestructura, garantizando trazabilidad completa.

La capacidad de *self-healing* de ArgoCD complementa directamente el mecanismo de *drift protection* de Kyverno: si alguien modifica manualmente una política en el clúster, ArgoCD detecta la desviación y la revierte al estado definido en Git; si alguien elimina una Network-Policy generada por Kyverno, el Background Controller de Kyverno la regenera. Ambas capas trabajan de forma coordinada para garantizar que el estado del sistema permanece siempre alineado con el definido en el repositorio.

5.2.4 GitHub Actions y Trivy

GitHub Actions actúa como motor de integración continua, ejecutándose automáticamente ante cada *push* al repositorio de aplicación [13]. El pipeline construye la imagen de contenedor, invoca Trivy para realizar un análisis de vulnerabilidades [14] y, únicamente si el análisis no detecta vulnerabilidades críticas, publica la imagen en el registro y actualiza la etiqueta de imagen en el repositorio de configuración.

La seguridad del propio pipeline merece atención específica. La alerta del tutor sobre el ataque del grupo TeamPCP, que comprometió pipelines CI/CD mediante la manipulación de GitHub Actions de terceros [17], motiva dos decisiones de diseño concretas que se detallan en la Sección 5.5: el anclaje de versiones de acciones mediante hash SHA y la limitación de permisos del token del runner al mínimo necesario.

5.2.5 Sysdig Secure

Sysdig Secure aporta la dimensión de seguridad en tiempo de ejecución (*runtime security*) al entorno, complementando los controles preventivos de Kyverno con detección de comportamiento anómalo durante la ejecución de los pods [15]. Como servicio SaaS, su integración con el entorno de laboratorio basado en máquinas virtuales puede presentar limitaciones técnicas, tal como se documenta en el Capítulo 4. No obstante, su rol arquitectónico es relevante: mientras Kyverno actúa en el momento del despliegue, Sysdig Secure actúa durante

la ejecución, cubriendo la superficie de ataque que no puede ser controlada estáticamente.

5.3 Flujo de despliegue DevSecOps

El flujo completo del sistema puede describirse en dos fases diferenciadas: la fase de integración continua, responsabilidad de GitHub Actions, y la fase de despliegue continuo, responsabilidad de ArgoCD y Kyverno. La separación explícita entre CI y CD es una decisión arquitectónica intencionada que se justifica en la Sección 5.5.

5.3.1 Fase de integración continua

El flujo de CI se inicia cuando un desarrollador realiza un *push* sobre la rama principal del repositorio de aplicación. GitHub Actions detecta el evento y lanza el workflow definido, que ejecuta secuencialmente los siguientes pasos:

En primer lugar, se construye la imagen de contenedor a partir del Dockerfile del repositorio. A continuación, Trivy analiza la imagen resultante en busca de vulnerabilidades conocidas, consultando su base de datos de CVEs. Si el análisis detecta vulnerabilidades de severidad crítica o alta, el pipeline falla en este punto y la imagen no es publicada, impidiendo que código vulnerable llegue al entorno de ejecución. Si el análisis es satisfactorio, la imagen se publica en el registro de contenedores con una etiqueta que incluye el hash del commit, garantizando la trazabilidad entre el artefacto desplegado y el código fuente que lo generó.

El último paso del pipeline de CI actualiza el manifiesto de despliegue en el repositorio de configuración, sustituyendo la etiqueta de imagen anterior por la nueva. Este commit en el repositorio de configuración actúa como señal de disparo para la fase de CD.

La Figura 5.2 ilustra este flujo completo de CI/CD.

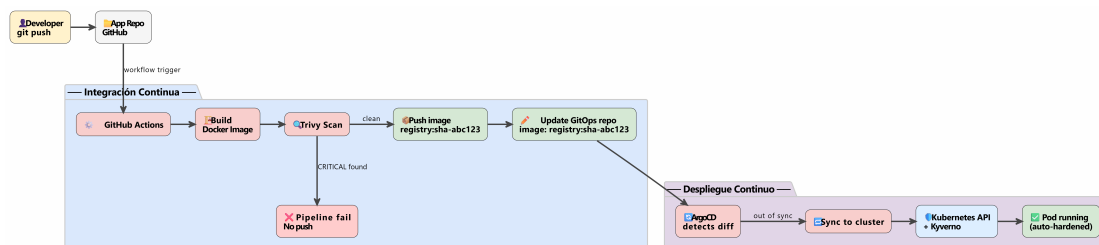


Figura 5.2: Flujo de integración y despliegue continuo con análisis de seguridad

5.3.2 Fase de despliegue continuo y admisión

ArgoCD monitoriza el repositorio de configuración de forma periódica. Al detectar el nuevo commit introducido por el pipeline de CI, computa la diferencia entre el manifiesto ac-

tualizado y el estado actual del clúster e inicia la sincronización, enviando la solicitud de actualización del recurso al API Server de Kubernetes.

En este punto interviene Kyverno a través del mecanismo de Admission Controllers. El proceso de admisión de Kubernetes sigue una secuencia determinada [8] que se ilustra en la Figura 5.3:

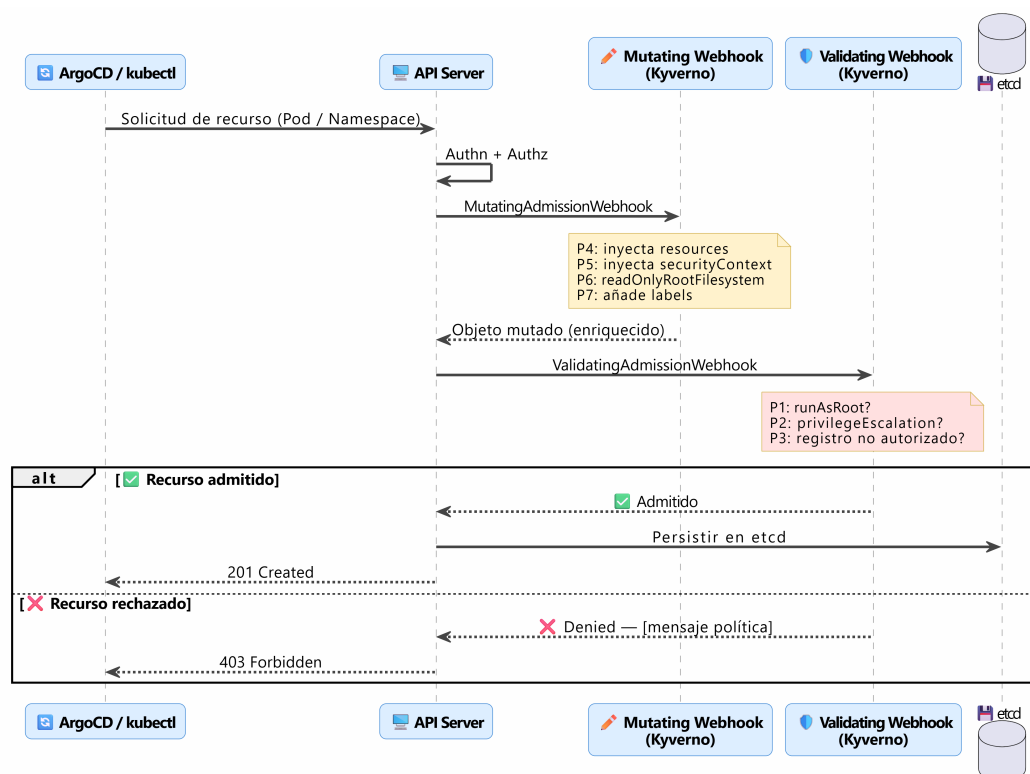


Figura 5.3: Flujo de admisión de Kyverno: mutación antes de validación

Tras la autenticación y autorización de la solicitud, el API Server invoca los *mutating admission webhooks* registrados. Kyverno recibe el objeto en su estado original y aplica secuencialmente todas las políticas de tipo *mutate* activas: inyecta los límites y solicitudes de recursos (P4), completa el `securityContext` del contenedor con las configuraciones seguras faltantes (P5), fuerza el sistema de ficheros de solo lectura (P6) y añade las etiquetas de trazabilidad (P7). El objeto devuelto al API Server es ya la versión enriquecida con todas las configuraciones de seguridad.

A continuación, el API Server invoca los *validating admission webhooks*. Kyverno evalúa las políticas de tipo *validate* sobre el objeto ya mutado. Dado que la mutación ha aplicado previamente las configuraciones seguras, la gran mayoría de pods pasan esta validación sin rechazo. Las políticas de validación actúan como última barrera para casos que la mutación no puede resolver: el uso de imágenes de registros no autorizados (P3), la ejecución explícita

como usuario root cuando el desarrollador lo ha especificado de forma deliberada (P1) o la solicitud explícita de escalada de privilegios (P2). Si alguna política de validación rechaza el objeto, el API Server devuelve un error con el mensaje definido en la política y el recurso no es persistido en etcd.

Si el objeto supera todos los webhooks de admisión, es persistido en etcd, el Scheduler determina en qué nodo debe ejecutarse y el kubelet del nodo worker procede con la creación del pod.

5.4 Arquitectura de red del clúster

La topología de red del entorno de laboratorio se articula en tres capas, tal como muestra la Figura 5.4.

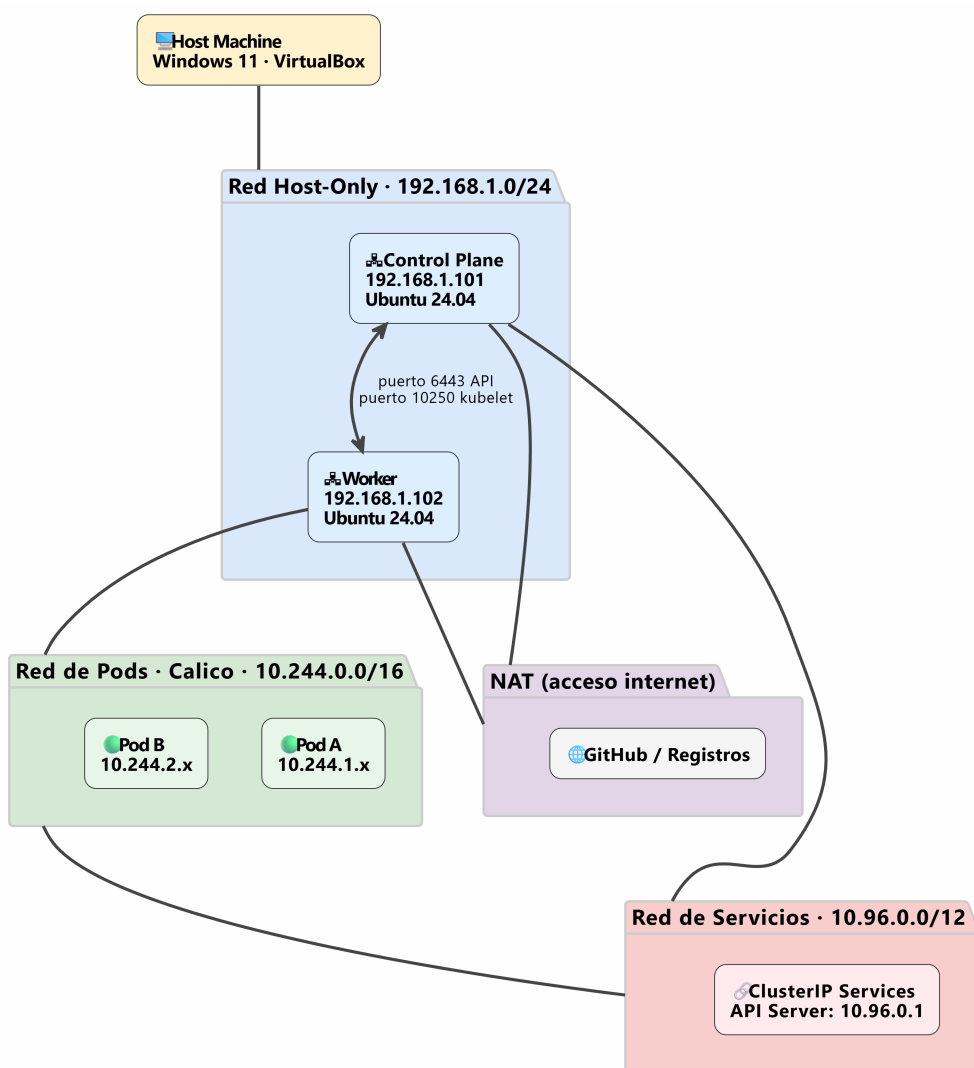


Figura 5.4: Arquitectura de red del clúster de laboratorio

La **capa de gestión** corresponde a la red *host-only* de VirtualBox ($192.168.1.0/24$), mediante la cual la máquina anfitriona y ambos nodos del clúster se comunican. Los nodos tienen direcciones IP estáticas asignadas: $192.168.1.101$ para el nodo de control y $192.168.1.102$ para el nodo worker. Toda comunicación interna del clúster —entre el API Server y los kubelets, entre el API Server y los controladores de Kyverno y ArgoCD— transita por esta red. Adicionalmente, cada nodo dispone de una interfaz NAT de VirtualBox que proporciona acceso a internet, utilizada para la descarga de imágenes de contenedor y la comunicación con servicios externos como GitHub o registros de imágenes.

La **capa de pods** está gestionada por Calico [16], que asigna un espacio de direccionamiento privado ($10.244.0.0/16$) a los pods del clúster. Calico implementa las Network Policies de Kubernetes mediante reglas de iptables o eBPF, en función de la configuración, garantizando que las restricciones de tráfico definidas en las políticas se aplican de forma efectiva. Esta implementación es un requisito para la política P8 del trabajo: la NetworkPolicy generada por Kyverno para bloquear el acceso al servicio de metadatos cloud ($169.254.169.254/32$) solo tiene efecto si el plugin CNI subyacente respeta y aplica las Network Policies de Kubernetes, condición que Calico cumple de forma nativa.

La **capa de servicios** utiliza el espacio de direccionamiento $10.96.0.0/12$, reservado por kubeadm para los objetos Service de tipo ClusterIP. Los servicios de los componentes del sistema —API Server ($10.96.0.1$), ArgoCD Server, Kyverno— son accesibles dentro del clúster a través de este espacio de direcciones, mientras que el acceso externo al API Server se realiza directamente a través de la IP del nodo de control mediante el puerto 6443.

5.5 Decisiones de diseño

Las decisiones arquitectónicas adoptadas en el diseño del sistema no son arbitrarias, sino que responden a requisitos técnicos y de seguridad concretos. Esta sección documenta las más relevantes y justifica cada una de ellas.

5.5.1 Separación entre CI y CD

La separación entre integración continua (GitHub Actions) y despliegue continuo (ArgoCD) es una decisión arquitectónica deliberada, en lugar de un único pipeline que construye y despliega en una sola secuencia. Esta separación ofrece varias ventajas. En primer lugar, desacopla las credenciales necesarias: el pipeline de CI necesita acceso al registro de imágenes y al repositorio de configuración, pero no al clúster Kubernetes; ArgoCD tiene acceso al clúster pero no al código fuente. Esta segregación reduce el radio de impacto de un posible compromiso de cualquiera de los dos componentes. En segundo lugar, permite aplicar

controles de seguridad en el límite entre ambas fases: la imagen solo es publicada si pasa el análisis de Trivy, y ArgoCD solo despliega lo que está en el repositorio de configuración, que actúa como punto de control auditado.

5.5.2 Alcance de ClusterPolicy frente a Policy

Kyverno permite definir políticas con alcance de clúster (`ClusterPolicy`) o con alcance de namespace (`Policy`). En este trabajo se utilizan exclusivamente `ClusterPolicy`, lo que garantiza que las políticas de seguridad se aplican a todos los namespaces del clúster sin excepciones. Una política namespaced podría ser inadvertidamente omitida al crear un nuevo namespace, creando una brecha de cobertura. El uso de `ClusterPolicy` implementa el principio de *secure by default*: la seguridad se aplica en todas partes a menos que sea explícitamente excluida mediante reglas de excepción documentadas.

5.5.3 Modo Enforce frente a Audit

Kyverno permite operar en dos modos: `Enforce`, que bloquea activamente los recursos que incumplen las políticas, y `Audit`, que permite el despliegue pero registra las infracciones en los `PolicyReports`. En este trabajo, las políticas de validación operan en modo `Enforce`, lo que garantiza que ningún recurso no conforme llega al clúster. Aunque en un entorno de producción real suele recomendarse comenzar en modo `Audit` para evitar interrupciones, en el contexto de este entorno de laboratorio el modo `Enforce` es el que permite demostrar de forma efectiva el mecanismo de bloqueo y su efecto real sobre los despliegues.

5.5.4 Mutación antes de validación como estrategia de hardening

El orden de ejecución de los webhooks de Kubernetes —mutación antes de validación— se aprovecha deliberadamente como estrategia de *hardening* automático. Las políticas de mutación no son complementos opcionales, sino el mecanismo principal de seguridad del sistema: en lugar de rechazar pods con configuraciones incompletas (lo que generaría fricción para los desarrolladores), Kyverno los completa automáticamente con el perfil de seguridad correcto. Las políticas de validación actúan únicamente sobre las configuraciones que no pueden resolverse mediante mutación, como el uso de registros de imágenes no autorizados o la presencia de privilegios explícitamente declarados.

Esta estrategia alinea seguridad y usabilidad: el desarrollador puede trabajar con manifiestos mínimos y obtener un pod correctamente securizado sin necesidad de conocer en detalle todos los campos de seguridad de Kubernetes. La responsabilidad del cumplimiento recae en la plataforma, no en el desarrollador.

5.5.5 synchronize: true en políticas de generación

La política P8 utiliza el campo `synchronize: true` en su definición de tipo *generate*. Esta configuración instruye al Background Controller de Kyverno para que monitorice los recursos generados y los recree automáticamente si son eliminados o modificados de forma no autorizada. Esta decisión implementa el principio de *drift protection* a nivel de recursos de seguridad: la NetworkPolicy que bloquea el acceso al IMDS no puede ser eliminada de forma permanente por un operador (intencionada o accidentalmente), ya que Kyverno la regenerará en cuestión de segundos. Sin este campo, la generación sería un evento único y la protección podría ser eludida.

5.5.6 Seguridad de la cadena de suministro del pipeline

El compromiso del grupo TeamPCP sobre pipelines CI/CD [17] ilustra que el propio pipeline de automatización puede convertirse en vector de ataque. En respuesta a esta amenaza, el diseño del pipeline adopta dos medidas concretas. La primera es el anclaje de versiones de las GitHub Actions utilizadas mediante su hash SHA completo, en lugar de etiquetas flotantes como `@latest` o `@v3`, que pueden ser redirigidas a versiones maliciosas sin modificar el nombre de la referencia. La segunda es la configuración del token de GitHub con permisos mínimos (*least privilege*): el runner solo dispone de los permisos estrictamente necesarios para su función, limitando el impacto de un posible compromiso del entorno de ejecución.

5.5.7 Elección de Calico como plugin CNI

La elección de Calico [16] frente a alternativas como Flannel o Weave responde a un requisito funcional directo: la política P8 requiere que las Network Policies de Kubernetes sean aplicadas efectivamente por el plugin CNI. Flannel, por ejemplo, no implementa soporte para Network Policies de forma nativa. Calico, en cambio, es uno de los plugins CNI de referencia en entornos de producción y proporciona una implementación completa del modelo de Network Policy de Kubernetes, garantizando que las reglas de bloqueo de tráfico generadas por Kyverno tienen efecto real sobre la comunicación de red de los pods.

Bibliografía

- [1] R. S. Pressman y B. R. Maxim, *Software Engineering: A Practitioner's Approach*. McGraw-Hill, 2019.
- [2] G. Kim, J. Humble, P. Debois y J. Willis, "The DevOps Handbook," 2016.
- [3] T. Sharma et al., "DevSecOps: A Multivocal Literature Review," 2020. dirección: <https://arxiv.org/abs/2007.12561>
- [4] A. Behl y K. Behl, "DevSecOps: Integrating Security into DevOps," 2017.
- [5] A. Rahman y L. Williams, "DevSecOps: Integrating Security into Continuous Delivery," 2019.
- [6] V. Mohan y L. Ben Othmane, "Towards DevSecOps: Integrating Security into DevOps," 2016. dirección: <https://ieeexplore.ieee.org/document/7488962>
- [7] B. Burns, B. Grant, D. Oppenheimer, E. Brewer y J. Wilkes, "Borg, Omega, and Kubernetes," *Communications of the ACM*, 2016. dirección: <https://dl.acm.org/doi/10.1145/2934556.2934564>
- [8] The Kubernetes Authors, *Kubernetes Documentation*, 2024. dirección: <https://kubernetes.io/docs/home/>
- [9] M. Hüttermann, *DevOps for Developers*. 2022.
- [10] Kyverno Project, *Kyverno Documentation*, 2024. dirección: <https://kyverno.io/>
- [11] OPA Project, *Open Policy Agent Documentation*, 2024. dirección: <https://www.openpolicyagent.org/>
- [12] Weaveworks, *GitOps Explained*, 2021. dirección: <https://www.weave.works/technologies/gitops/>
- [13] GitHub, *GitHub Actions Documentation*, 2024. dirección: <https://docs.github.com/en/actions>
- [14] Aqua Security, *Trivy Documentation*, 2024. dirección: <https://aquasecurity.github.io/trivy/>
- [15] Sysdig, *Sysdig Secure*, 2024. dirección: <https://sysdig.com/products/secure/>

-
- [16] Tigera, *Calico Documentation*, 2024. dirección: <https://docs.tigera.io/calico/latest/about/>
 - [17] Socket Security, *TeamPCP Partners with Vect Targeting OSS Supply Chains*, Accessed: April 2026, 2026. dirección: <https://socket.dev/blog/teampcp-partners-with-vect-targeting-oss-supply-chains>
 - [18] Argo Project, *ArgoCD Documentation*, 2024. dirección: <https://argo-cd.readthedocs.io/>